

ECU-25 Explained — A Detailed Walkthrough of the Design

Companion to: <docs/superpowers/specs/2026-07-29-genset-ecu-design.md> **Audience:** B.Tech ECE graduate. Assumes you know basic circuit theory, op-amps, microcontrollers, and C programming — but not automotive electronics, power electronics tricks, or genset practice. Everything beyond that is explained here.

1. What We Are Building, In Plain Words

A **genset controller** is the box on the front of a diesel generator panel. It has one job with many parts:

1. **Start and stop the engine** safely and automatically.
2. **Protect** the engine and alternator — shut down before something expensive breaks.
3. **Measure** everything: engine sensors, battery, and the 415 V 3-phase output.
4. **Transfer the load** — when utility mains fails, start the genset and switch the building onto it; when mains returns, switch back and stop the engine. This automatic behaviour is called **AMF: Auto Mains Failure**.
5. **Talk** to the outside world — a display for humans, Modbus for SCADA/BMS systems, CAN/J1939 for engine electronics.

Commercial examples: Deep Sea Electronics DSE 4520, ComAp IntelliLite, Smartgen HGM6120. Ours is called **ECU-25** (ECU for a 25 kVA genset). It is an **original design** — same feature class as those products, but every circuit is designed by us from first principles. Nothing is copied, so we can legally build, modify, and manufacture it, and more importantly we understand every part.

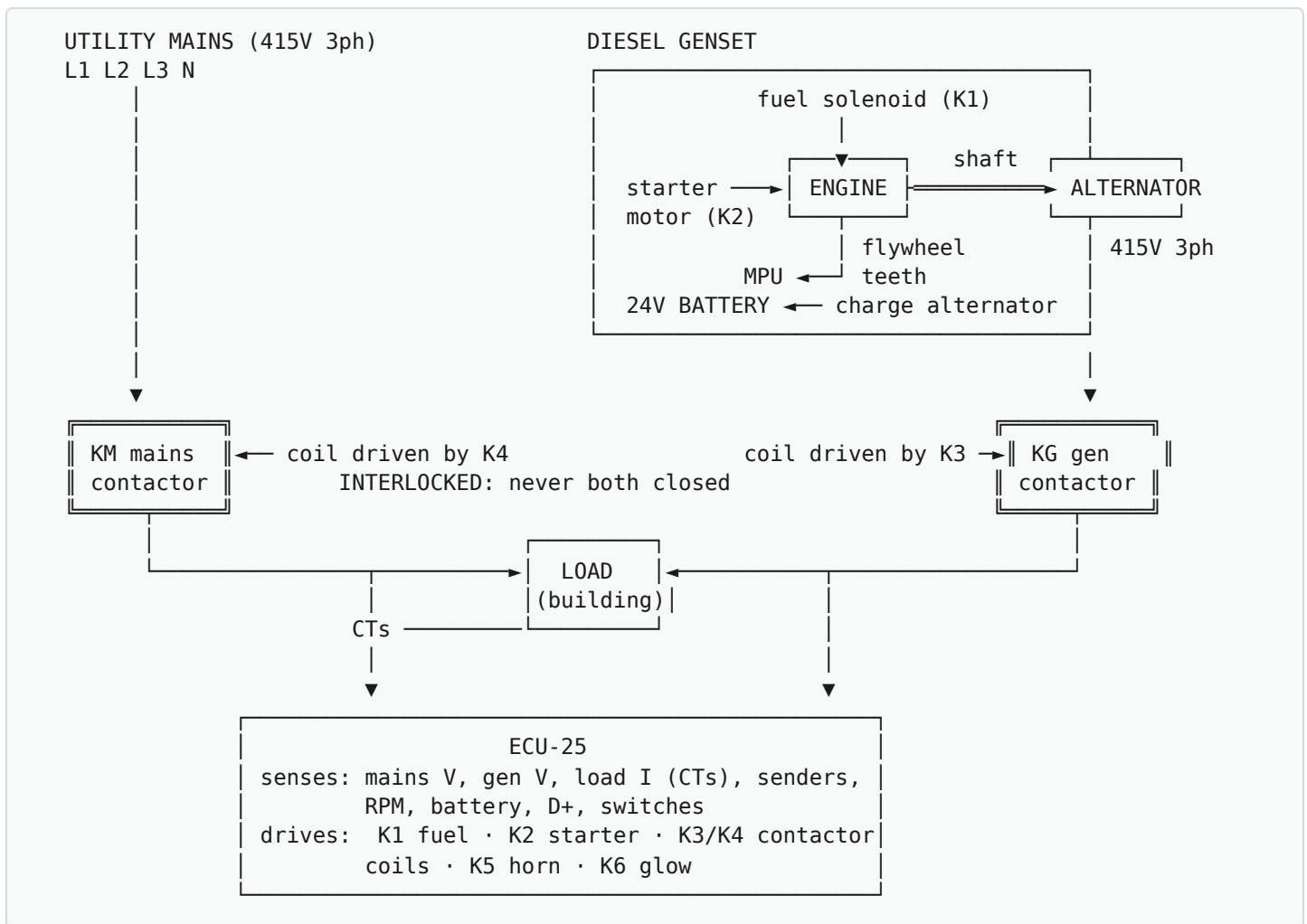
The machine it controls

A **25 kVA diesel genset** is:

- A **diesel engine** (~30–40 HP) with a **mechanical governor** — a flyweight mechanism inside the fuel pump that holds the engine at 1500 RPM by itself. Because the governor is mechanical, our controller does **not** control speed. It only controls **fuel on/off** (a solenoid valve) and the **starter motor**.
- An **alternator** bolted to the engine, producing **415 V line-to-line, 3-phase, 50 Hz** when the engine turns at 1500 RPM. (Frequency is locked to speed: a 4-pole alternator gives 50 Hz at exactly 1500 RPM. This is why speed and frequency protections are really the same thing measured two ways.)
- A **24 V battery system** — battery, starter motor, and a small **charge alternator** (like a car alternator) that recharges the battery while the engine runs.

Design update (2026-08-11): the target engine is confirmed to be a **common-rail electronic diesel** with its own engine-management ECU (the box that controls injection, rail pressure, EGR, DPF). That changes ECU-25's role from "drives the engine directly" to "**supervises the engine over J1939**": start/stop via a run-enable relay + CAN, engine speed / oil pressure / coolant temp / fault codes read from the engine ECU's broadcasts, and engine fault codes (DM1) shown on our display. Everything else — AC metering, protections, AMF transfer, display, Modbus — is unchanged. The analog sender inputs, MPU input, and fuel-solenoid relay described below stay in the hardware as a **legacy mode**, so the same board still runs older mechanical-governor gensets. Where the text below says "fuel solenoid", read "run enable" for the electronic engine.

The complete installation — single-line view



One picture, whole job: watch both power sources, run the engine, and decide which contactor feeds the load.

2. System Architecture — Why Two Boards

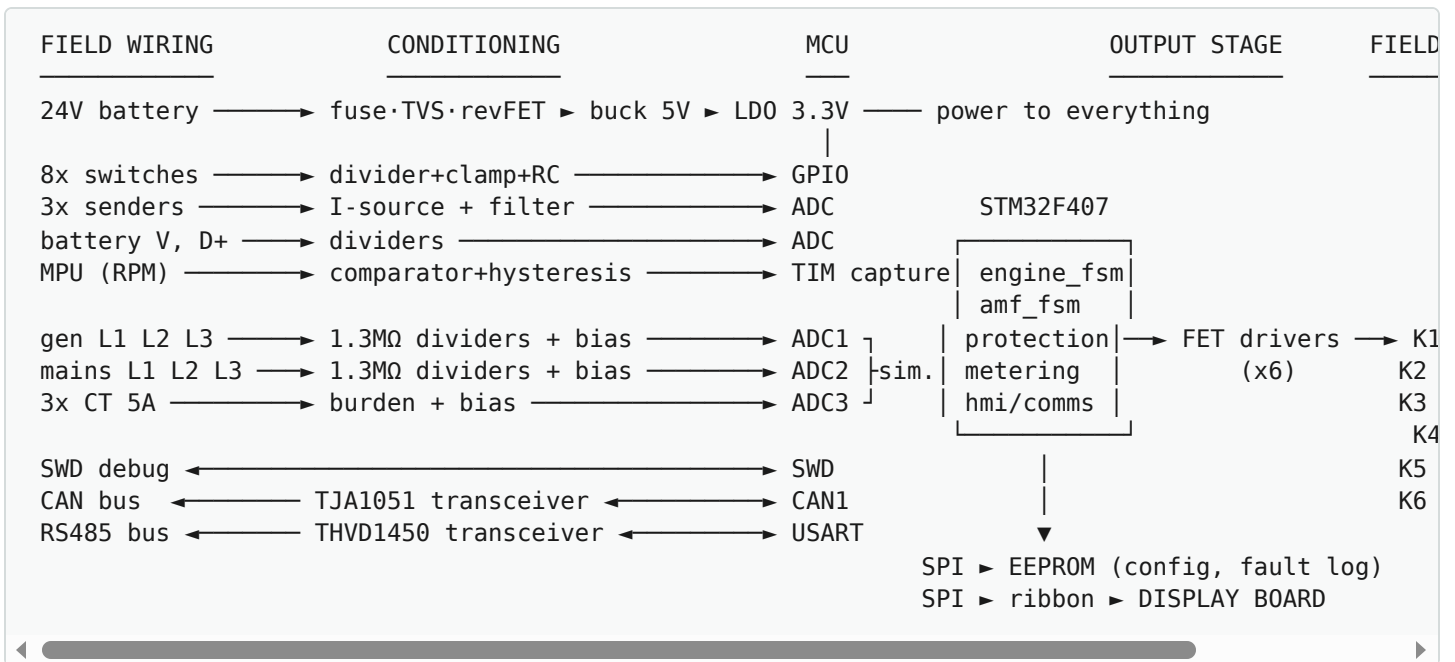
DISPLAY BOARD (2-layer) 4.3" 480x272 color TFT + RA8875 (SPI) Keys: STOP AUTO MANUAL START ▲ ▼ ↻ (menu) 8 status LEDs	20-way ribbon (SPI + GPIO, 3.3 V + 5 V)
---	---

MAIN BOARD (4-layer, ~120 x 100 mm) Power supply STM32F407VGT6 EEPROM 8x digital in 3x sender in RPM in 6x AC voltage in 3x CT in 6x relay CAN (J1939) RS485 (Modbus) SWD

Why split? The main board carries 415 V nets, relay contacts, and a switching power supply — it wants to live deep in the panel with fat wires. The display wants to be on the panel door at eye level. Splitting them means: each layout is simpler, the display can be swapped/upgraded without respinning the main board, and only logic-level signals cross the ribbon cable (no hazardous voltage goes to the door). This is exactly how commercial controllers are built internally.

Why 4 layers on the main board? A PCB can have 2, 4, 6+ copper layers. With 4 layers we dedicate one inner layer to a solid **ground plane** and one to power. A continuous ground plane gives every signal a low-inductance return path directly underneath it. That matters here because this one board mixes: a switching converter (fast dV/dt edges), precision analog measurement (millivolt-level accuracy wanted), relay switching (arcs and coil kickback), and mains-frequency high voltage. On a 2-layer board, return currents share long looping paths, they couple into each other, and the analog readings get noisy. Four layers is the cheapest robust answer; commercial ECUs use 4–8.

Main board — internal signal flow



Read it left to right: raw field signals enter, each gets tamed by its conditioning circuit, the MCU decides, decisions leave through relay contacts. Every section of this document below explains one row of this picture.

3. Power Supply — Surviving the 24 V Vehicle Environment

The battery rail on an engine is one of the nastiest electrical environments in electronics. The supply chain is:

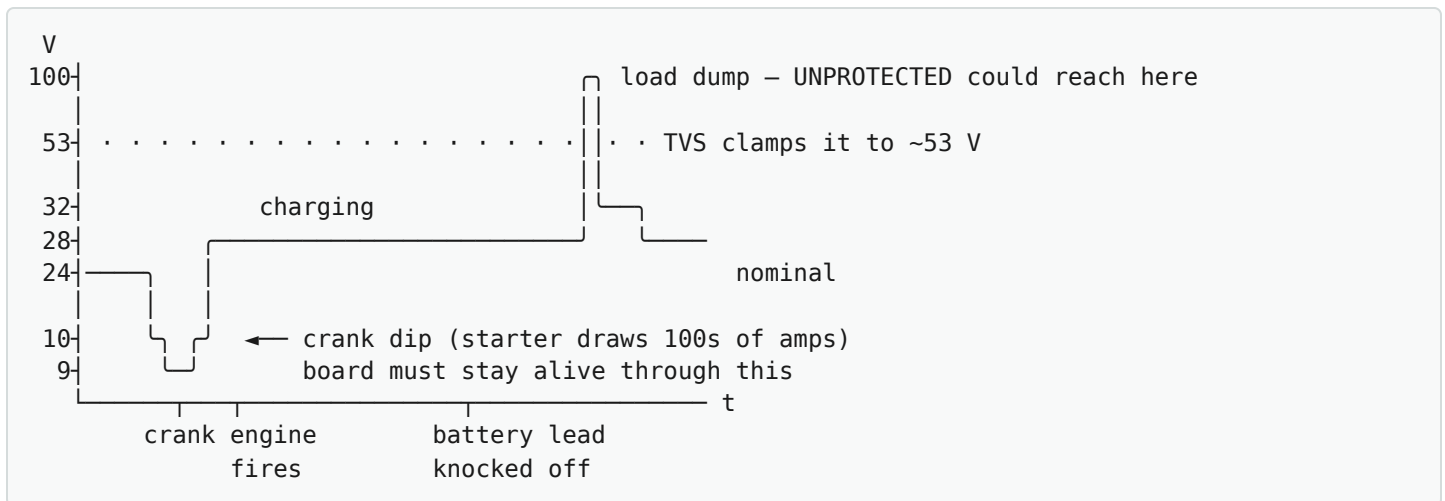
24V battery → fuse → TVS clamp → reverse-polarity MOSFET → ferrite + bulk caps
 → LM5164-Q1 buck (24V → 5V) → TLV75533 LDO (5V → 3.3V)

3.1 What the input must survive

- **Cranking dips.** When the starter motor engages, it draws hundreds of amps and the "24 V" rail sags — briefly down to 9–10 V. If our 5 V and 3.3 V rails collapse during that dip, the MCU resets mid-start-sequence — unacceptable. So the input range is **9-32 V continuous**, and a bulk electrolytic capacitor stores enough charge to ride through the worst milliseconds.
- **Load dump.** The classic automotive fault: the battery cable falls off (or is disconnected) while the charge alternator is charging hard. The alternator's field can't collapse instantly, so the rail flies up — in a 24 V system, transients approaching **60-100 V for tens of milliseconds**. Anything not designed for this dies.
- **Reverse battery.** A mechanic connects the battery backwards. It happens constantly in the field. The design must block it harmlessly.
- **Inductive spikes.** Every solenoid, relay coil, and injector on the machine kicks voltage spikes back onto the rail when switched off.

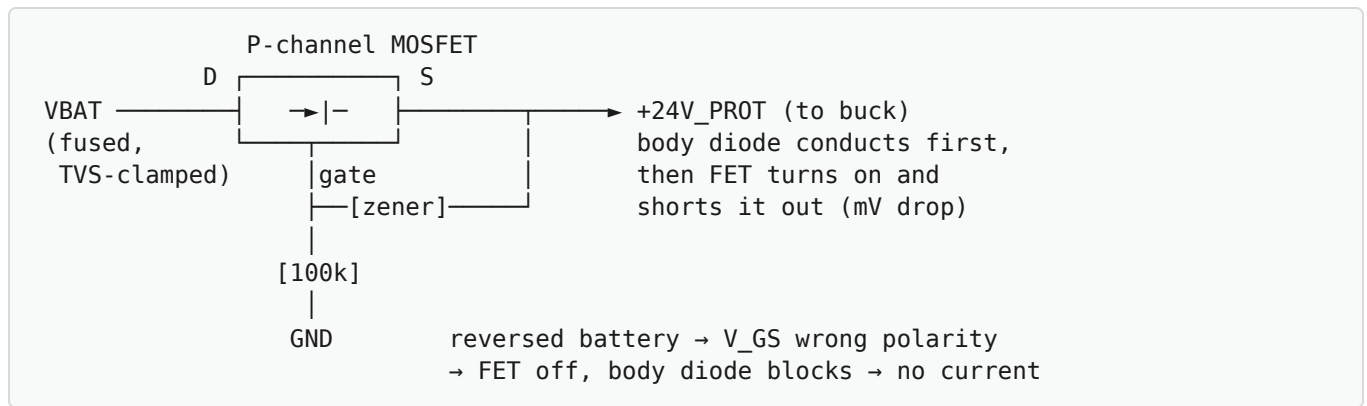
Relevant standard (for awareness, not certification): **ISO 7637-2** defines these transient pulses formally.

What the "24 V" rail actually looks like over a start/stop cycle:



3.2 Each protection element

- **Fuse (5 A blade type).** Last-resort protection: if something on the board fails short, the fuse opens before the wiring harness catches fire. It protects the *wiring*, not the electronics — fuses are far too slow to save semiconductors.
- **TVS diode (SMCJ33CA).** A **Transient Voltage Suppressor** is a purpose-built avalanche diode. Below its standoff voltage (33 V) it is invisible. When a transient exceeds its breakdown (~36–40 V) it avalanches and clamps the rail (clamping ~53 V at rated pulse current), absorbing hundreds of watts for milliseconds. It's the component that eats the load dump. "CA" = bidirectional version (also clamps negative spikes). SMC package = the physically large version, because transient energy absorption scales with die size.
- **Reverse-polarity P-channel MOSFET.** The textbook answer is a series diode — but a diode drops ~0.7 V continuously, wasting power and reducing headroom during crank dips. The production trick: a **P-MOSFET with source toward the load, gate pulled to ground**. With correct battery polarity, the gate is ~24 V below the source, the FET turns fully on, and drop is just $I \times R_{DS(on)}$ — millivolts. With reversed battery, the gate-source voltage is the wrong polarity, the FET stays off, and (with the body diode oriented to block) no current flows. A zener protects the gate from exceeding $\pm V_{GS(max)}$.



- **Ferrite bead + capacitors.** A ferrite bead is a lossy inductor: near-zero resistance at DC, but it turns high-frequency noise (MHz+) into heat. Combined with capacitors on both sides it forms a low-pass π -filter — keeping engine electrical noise out of the board, and keeping our buck converter's switching noise from radiating back up the battery cable (an EMC requirement).

3.3 The buck converter — LM5164-Q1

We need 5 V from 24 V. Two ways to do that:

- **Linear regulator:** drops $(24-5) = 19$ V across itself at full current. At 300 mA that's ~ 5.7 W of pure heat. Not viable.
- **Buck (step-down switching) converter:** switches the input on and off at high frequency through an inductor. The inductor and output capacitor average the chopped waveform to a smooth DC at the target voltage; energy is *transferred*, not burned. Efficiency 85–92 %.

Refresher on buck operation: a high-side switch connects V_{IN} to the inductor for duty-cycle fraction D of each cycle; $V_{OUT} \approx D \times V_{IN}$. When the switch opens, inductor current keeps flowing (inductors resist current change) through a low-side path (a diode, or in a **synchronous** buck like ours, a second internal MOSFET — more efficient). The control loop adjusts D continuously to regulate V_{OUT} .

Why the LM5164-Q1 specifically:

- **100 V maximum input.** Our TVS clamps load dump at ~ 53 V. The converter must survive that with margin — a "36 V max" converter would be at its edge; a 100 V part shrugs.
- **Synchronous, internal FETs** — few external parts, good efficiency.
- **-Q1 suffix = AEC-Q100 automotive qualification.** The part is tested and guaranteed for automotive stress: -40 to $+125$ °C, temperature cycling, humidity, vibration lifetime tests. On battery-connected circuits we prefer Q100 parts throughout.

3.4 The LDO — TLV75533, 5 V → 3.3 V

A buck is efficient but **noisy** — every switching edge leaves ripple (tens of mV at the switching frequency and its harmonics) on its output. Our ADC measures signals where $1 \text{ LSB} \approx 0.8 \text{ mV}$; the ADC reference must be clean. So the 3.3 V rail comes from a **low-dropout linear regulator (LDO)** fed by the 5 V rail. An LDO is a linear regulator whose pass element can regulate with very little headroom ("dropout"), and a good one has high **PSRR** (power-supply rejection ratio) — it actively

attenuates the ripple arriving at its input. Dropping only 5→3.3 V at modest current, the heat is trivial, and the output is quiet enough for analog work.

Rail budget: 5 V feeds relay driver logic, sender pull-up sources, LCD backlight. 3.3 V feeds MCU, CAN and RS485 transceivers, EEPROM, and all analog front ends.

4. The Microcontroller Core — STM32F407VGT6

The **STM32F407VGT6** is an ST Microelectronics ARM **Cortex-M4** MCU: 168 MHz, 1 MB flash, 192 KB RAM, hardware FPU, LQFP-100 package. Why this one:

- **Triple ADC.** It has **three independent 12-bit ADCs that can sample simultaneously**. This is the killer feature for us: to compute 3-phase power correctly you must sample voltage and current of a phase *at the same instant* — sequential sampling introduces artificial phase shift, which corrupts the power-factor and kW math. Triple-ADC "simultaneous regular" mode samples three channels on one trigger.
- **Two CAN peripherals (bxCAN)** — J1939 needs one; a second is spare.
- **Plenty of timers** — input capture for RPM measurement, PWM for the future governor output.
- **FPU** — RMS and power math uses floating point comfortably at 168 MHz.
- Mature, cheap, enormous community knowledge base — right choice for a learning build.

4.1 Support circuitry around the MCU

- **8 MHz HSE crystal.** The MCU's internal RC oscillator is $\pm 1\%$ -ish over temperature — fine for blinking LEDs, not for CAN bit timing or accurate frequency measurement. An external crystal is $\sim \pm 30$ ppm. The PLL multiplies 8 MHz up to 168 MHz.
- **32.768 kHz LSE crystal + supercapacitor.** Runs the **RTC (real-time clock)** so fault-log entries carry real timestamps. $32\,768 = 2^{15}$, so a simple binary counter divides it to exactly 1 Hz — the universal watch-crystal frequency. The supercapacitor keeps only the tiny V_BAT domain (RTC + backup registers) alive for days when the board is unpowered.
- **Decoupling capacitors.** One 100 nF ceramic at every VDD pin, closest possible. Refresher on why: when the CPU switches, it demands current in sub-nanosecond bursts; the supply wiring has inductance and cannot deliver instantaneously. The local capacitor is a tiny charge reservoir next to the pin. Plus one bulk 4.7–10 μ F per rail region.
- **NRST + BOOT0.** NRST (reset) gets an RC and a test point. BOOT0 strapped low = boot from flash; a jumper to high invokes the built-in ROM bootloader (firmware recovery over UART without a debugger).
- **SWD header. Serial Wire Debug,** the 2-wire ARM debug interface (SWDIO + SWCLK), on a TC2030 tag-connect footprint (6 pads on the board, spring-pin cable, no fitted connector). This is how we flash and live-debug with an ST-Link.
- **IWDG — independent watchdog.** A hardware down-counter on its own internal oscillator. Firmware must "kick" it periodically; if firmware hangs, the counter expires and hard-resets the

chip. On a machine that controls a diesel engine, a hung controller must never stay hung. (See §10 for how we kick it honestly.)

4.2 EEPROM — M95M02-DR

256 KB SPI EEPROM storing **configuration** (all user settings) and the **fault log**. Why external EEPROM instead of the MCU's flash? MCU flash erases in large sectors, wears out sooner (~10 k cycles), and writing it stalls the CPU. EEPROM writes byte-wise, endures ~4 million cycles, and is immune to firmware-update erases. Config is stored with a **CRC** (cyclic redundancy check — a checksum) and a version number, so corrupted or outdated data is detected and defaults are loaded instead of garbage.

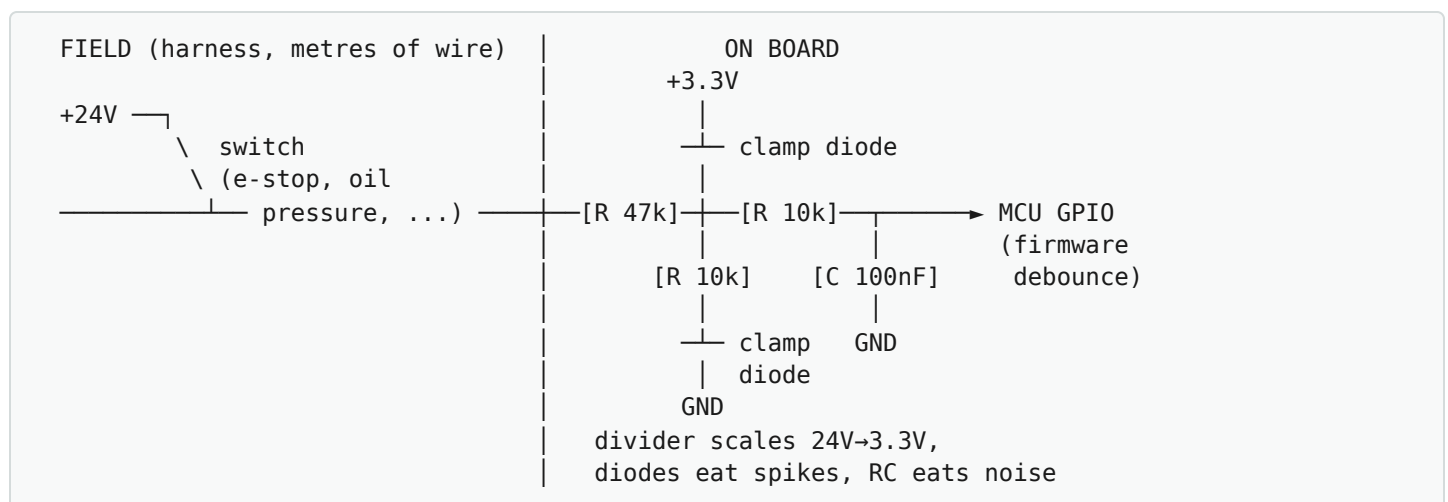
5. Digital Inputs (8 channels)

These read **switches**: contacts that are either open or closed. On the engine they are things like the emergency-stop mushroom button, the low-oil-pressure switch (closes when pressure is dangerously low), the high-coolant-temperature switch, a remote-start command from a BMS, a coolant-level float switch.

Why not connect a switch straight to an MCU pin? Because the wire to that switch may be 5 metres long, running beside the starter cable through an engine bay. It will carry 24 V levels (logic thresholds far above noise), pick up spikes, and bounce. Each channel therefore has:

1. **Resistor divider** — scales 24 V down to logic level, and sets a wetting current through the switch contacts (a few mA keeps mechanical contacts clean of oxide film).
2. **Clamp diodes** — Schottky diodes to the rail and ground, so any spike beyond the rails is diverted harmlessly.
3. **RC low-pass filter (~1 ms)** — absorbs fast noise bursts.
4. **Firmware debounce** — a mechanical contact physically bounces for a few ms when it closes; firmware requires N consecutive identical samples before accepting a change.

One channel, end to end:



Configurable active-high/low: some senders switch to battery positive, some switch to ground, depending on the engine's wiring convention. Firmware lets each input's polarity and function be assigned in the config menu — this is what makes a controller "universal" across engine brands.

6. Analog Sender Inputs (3 channels) + DC Measurements

6.1 What a "sender" is

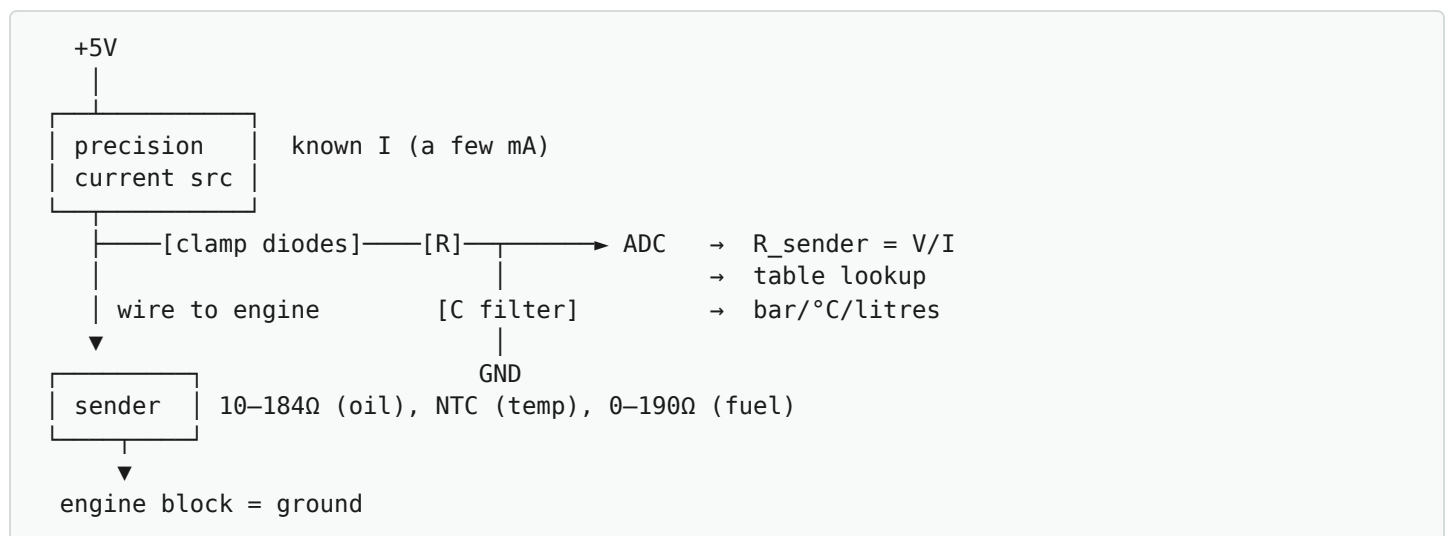
Engine instrumentation sensors are called **senders** (the name comes from "sending unit" for a dashboard gauge). They are simply **variable resistors**:

- **Oil pressure sender:** a diaphragm moves a wiper on a resistive track. VDO standard curve: $10\ \Omega$ at 0 bar $\rightarrow$ $184\ \Omega$ at 10 bar.
- **Coolant temperature sender:** an **NTC thermistor** — resistance falls as temperature rises, non-linearly (roughly exponentially).
- **Fuel level sender:** a float arm moving a wiper, typically $0\text{--}190\ \Omega$ empty-to-full (or reverse).

6.2 How we read a resistance

Feed the sender a **known current** from a precision current source (derived from the 5 V rail), and measure the voltage across it with the ADC: $R = V/I$. Why a current source instead of a simple pull-up resistor? With a plain pull-up, the transfer curve $V(R)$ is nonlinear and its slope depends on the pull-up's tolerance and the 5 V rail accuracy. A current source makes V directly proportional to R — easier math, better accuracy at the low- Ω end where the oil-pressure curve lives.

Each channel also gets an RC anti-alias filter and clamp diodes (the wire to a sender can short to battery positive in a chafed harness — the input must survive 24 V indefinitely).



Curves in firmware: the resistance $\rightarrow$ pressure or resistance $\rightarrow$ temperature relationship is stored as an interpolation table in config. Different engine brands use different sender curves; making the table editable makes the hardware universal.

6.3 Battery voltage and charge-fail detection

- **Battery voltage:** a protected resistive divider to an ADC channel. Used for battery high/low alarms and to compensate other measurements.
 - **Charge alternator D+ terminal:** the charge alternator (the small 24 V one, not the 415 V main alternator) needs a small **excitation current** fed into its D+ terminal to self-start generating; in a car this comes through the charge-warning lamp. We provide that excitation through a resistor from a switched supply, and monitor D+ voltage with a divider. Engine running but D+ low $\Rightarrow$ **charge failure** alarm (broken belt, dead alternator) — a classic genset protection, because a genset that flattens its own battery cannot restart in the next power cut.
-

7. RPM Input — Magnetic Pickup

7.1 The sensor

An **MPU (magnetic pickup unit)** is a coil wound around a permanent-magnet core, threaded into the flywheel housing so its tip sits $\sim 0.5\text{--}1$ mm from the ring-gear teeth. Each passing steel tooth changes the magnetic flux through the coil, inducing a voltage pulse (Faraday's law). No power supply needed. Output is roughly sinusoidal; its **amplitude varies enormously with speed** — maybe 0.5 V peak at cranking speed, tens of volts at rated speed. Frequency = teeth $\times$ revolutions per second.

7.2 Conditioning — why a comparator with hysteresis

The MCU timer needs clean logic edges. The raw MPU signal is a variable-amplitude sine with noise. So:

1. **AC-couple** the input (removes any DC offset) and clamp it to safe levels.
2. Feed it to an **LM2903 comparator** configured with **hysteresis** (a Schmitt trigger): the switching threshold moves apart in the two directions, e.g. rise above +100 mV to go high, fall below -100 mV to go low. Without hysteresis, noise riding on the signal near the threshold produces bursts of false edges — the RPM reading would jump around and could falsely trigger overspeed. With hysteresis, one tooth = exactly one clean edge.
3. The comparator output goes to an STM32 **timer input-capture** channel. The timer timestamps each edge in hardware; firmware computes the period between edges $\rightarrow$ frequency $\rightarrow$ RPM = (edge frequency $\div$ number of flywheel teeth) $\times$ 60. Tooth count is a config parameter.

flywheel teeth ► MPU coil ► AC-couple ► clamp ► LM2903 + hysteresis ► TIM capture

raw MPU signal (amplitude grows with speed):

cranking...

small

~~~~~

...rated speed

tens of volts

upper threshold  
lower threshold

+100 mV  
-100 mV

} hysteresis band:  
noise can't retrigger

comparator out:

—□—□—

—□—□—

exactly one clean edge per tooth  
period between edges → RPM

**W-terminal alternative:** the charge alternator's "W" terminal outputs a pulse train proportional to engine speed (from one stator phase). It is a cheaper RPM source (no MPU to buy) but less precise. A jumper lets either source use the same conditioning path.

**Why RPM matters so much:** crank disconnect (detect "engine has fired, release the starter" — engaging a starter into a running engine destroys the starter pinion), underspeed/overspeed protection, and a sanity cross-check against generator frequency.

## 8. AC Voltage Sensing (6 channels) — Measuring 415 V with a 3.3 V ADC

We measure six line-to-neutral voltages: generator L1, L2, L3 and mains L1, L2, L3. In a 415 V line-to-line system, line-to-neutral is  $415/\sqrt{3} \approx 240$  V RMS, i.e.  $\pm 340$  V peak; we design for 300 V RMS to have margin.

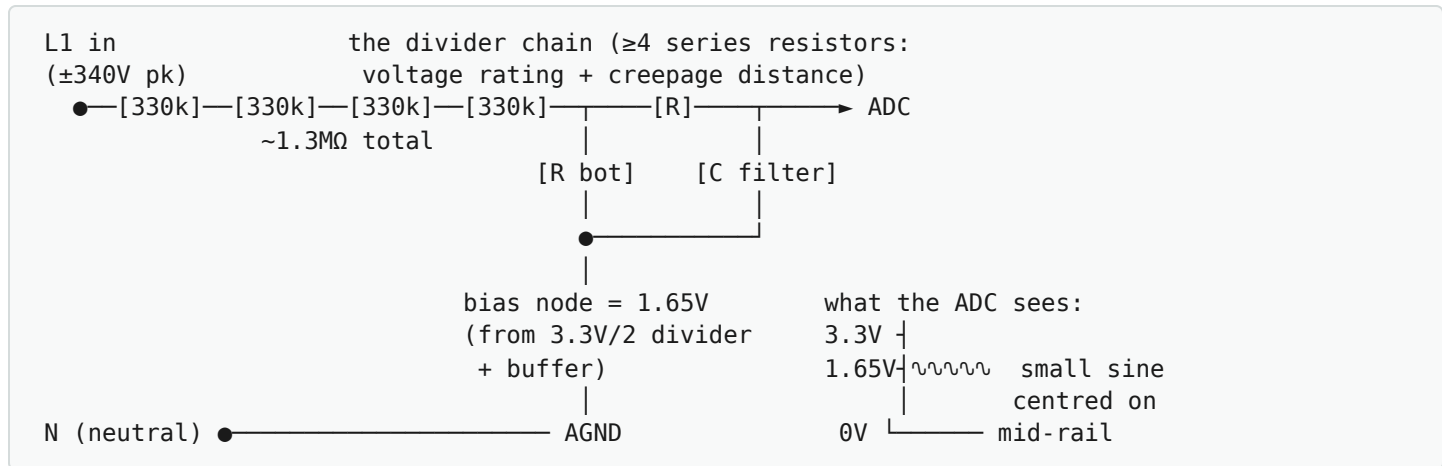
### 8.1 The high-impedance divider — the industry-standard trick

Each channel is a **resistive voltage divider with  $\sim 1.3$  M $\Omega$  total top resistance**, scaling  $\pm 340$  V peak down to the ADC's range. Design details that matter:

- **Series chain of  $\geq 4$  resistors (1206 size)** rather than one: each SMD resistor has a maximum working voltage (a 1206 is typically rated  $\sim 200$  V); four in series share the 340 V peak safely, and the physical length of the chain provides **creepage distance** (see below).
- **$\sim 1.3$  M $\Omega$  means  $\sim 0.18$  mA flows** — the divider dissipates  $\sim 45$  mW and presents negligible load. There is **no galvanic isolation**: the board's ground is referenced to the AC neutral through the measurement network. This is exactly how DSE/ComAp-class controllers do it — it is safe *if* creepage/clearance rules are respected and the enclosure/terminals prevent contact. (The expensive alternative — voltage transformers or isolation amplifiers per channel — adds cost and drift, and industry practice omits it at this product class.)
- **Mid-rail bias**: the AC signal swings negative, but the ADC only accepts 0–3.3 V. So the divided signal is superimposed on a **1.65 V DC bias** (half rail). The ADC sees a small sine centred on 1.65 V; firmware subtracts the offset.

- **RC anti-alias filter:** before any sampled system, frequencies above half the sampling rate must be attenuated or they **alias** — fold back and masquerade as low frequencies (Nyquist). A simple RC with cutoff well above 50 Hz but well below the sampling rate suffices.

One AC channel (same circuit ×6):



## 8.2 Creepage, clearance, and layout separation

Two safety distances govern high-voltage PCB design:

- **Clearance** — shortest distance through *air* between two conductors.
- **Creepage** — shortest distance *along the board surface*, which matters because dust and humidity make surfaces slightly conductive over time.

For 300 V RMS working voltage in a normal pollution environment, we keep  **$\geq 3$  mm** between AC nets and logic, place all AC terminals on one board edge, and keep the entire AC region out of the ground-plane area (plane cut-outs under the dividers). Firmware cannot fix a creepage violation; this is a layout law.

## 8.3 What firmware computes from these channels

Per channel, from the sampled waveform (see §12.4 for the sampling scheme): **true RMS voltage**, **frequency** (zero-crossing timing with interpolation), and phase relationships. From these: over/under-voltage and over/under-frequency protections for the generator, and the "is mains healthy?" decision for AMF.

**Why "true RMS" is emphasized:** cheap meters measure the rectified average and scale it assuming a perfect sine. Generator waveforms under nonlinear load (rectifier loads, UPS front-ends) are *not* perfect sines, and the average-scaled shortcut misreads them. True RMS =  $\sqrt{\text{mean of } v^2}$  over a whole number of cycles — correct for any waveshape.

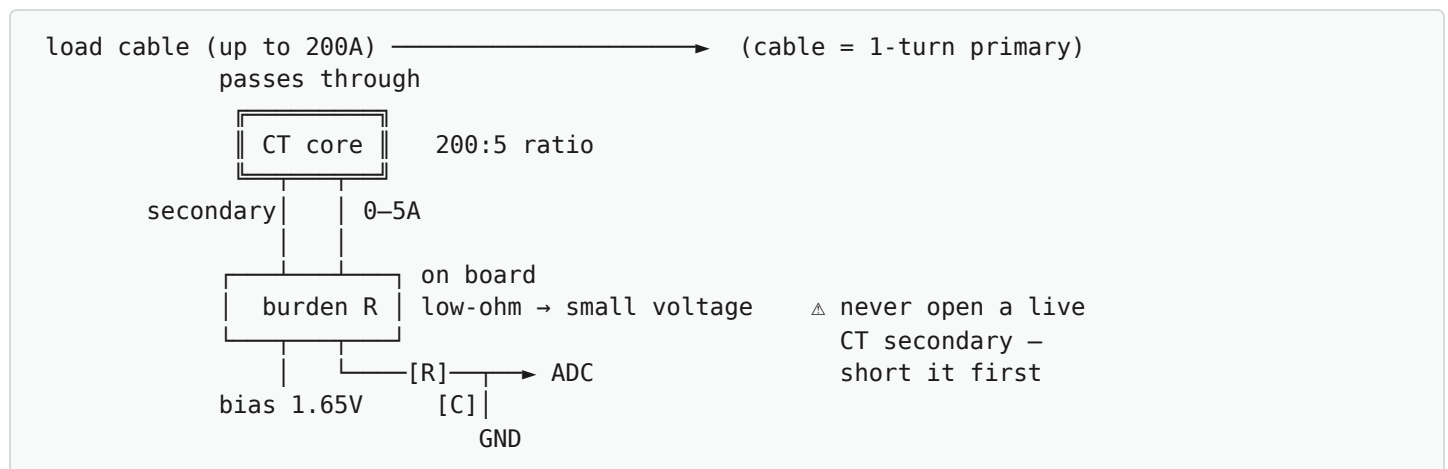
## 9. Current Transformer Inputs (3 channels)

### 9.1 CT theory in one paragraph

A **current transformer** is a toroidal magnetic core clipped around a power cable. The cable itself is the 1-turn primary; the CT's N-turn secondary delivers a current of  $I_{\text{primary}}/N$ . A "200/5 CT" outputs 5 A of secondary current when 200 A flows in the cable. Crucially, a CT is a *current* source: its secondary must always drive a low-impedance load called the **burden**. **An open-circuited CT secondary under load develops dangerous high voltage** (the transformer tries to force its current through infinite impedance) — this is the one famous CT safety rule: never open a live CT circuit; short it first.

### 9.2 Our input channel

Per channel: the 5 A secondary passes through a **low-ohm burden resistor** on the board, converting current to a small voltage (a fraction of a volt at rated current — keeps burden dissipation and CT accuracy comfortable); this voltage is biased to the 1.65 V mid-rail, RC-filtered, and sampled. The **CT ratio** (e.g. 200/5) is a config parameter; firmware multiplies back to real amps.



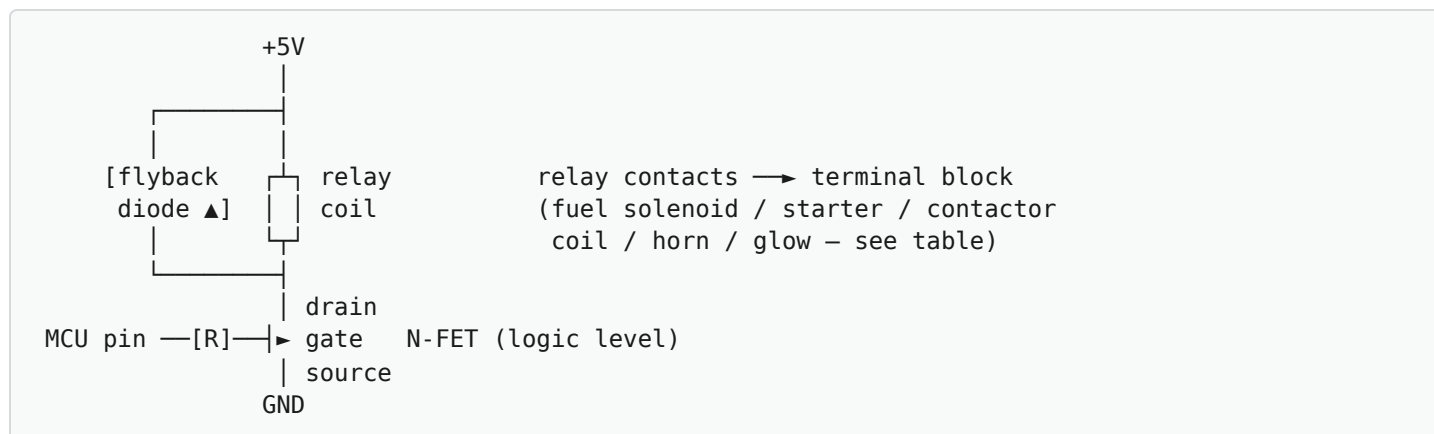
### 9.3 What firmware computes

Sampling each phase's voltage and current **simultaneously** (the triple-ADC feature), firmware computes: RMS current per phase, **active power**  $P = \text{mean}(v \cdot i)$ , **apparent power**  $S = V_{\text{RMS}} \cdot I_{\text{RMS}}$ , **power factor**  $PF = P/S$ , kW/kVA totals, and accumulated **kWh**. Plus overcurrent protection with a time delay (short overloads are normal — motor starting — sustained overload is not).

## 10. Relay Outputs (6 channels)

Six electromechanical relays, each driven by a logic-level N-channel MOSFET from an MCU pin, each coil with a **flyback diode**.

**Flyback refresher:** a relay coil is an inductor. When the driving FET switches off, the coil current cannot stop instantly ( $V = L \cdot di/dt$ ); the collapsing field drives the FET's drain to destructive voltages. A diode across the coil gives that current a circulating path, clamping the spike to one diode drop. Non-negotiable on every coil.



The assignments:

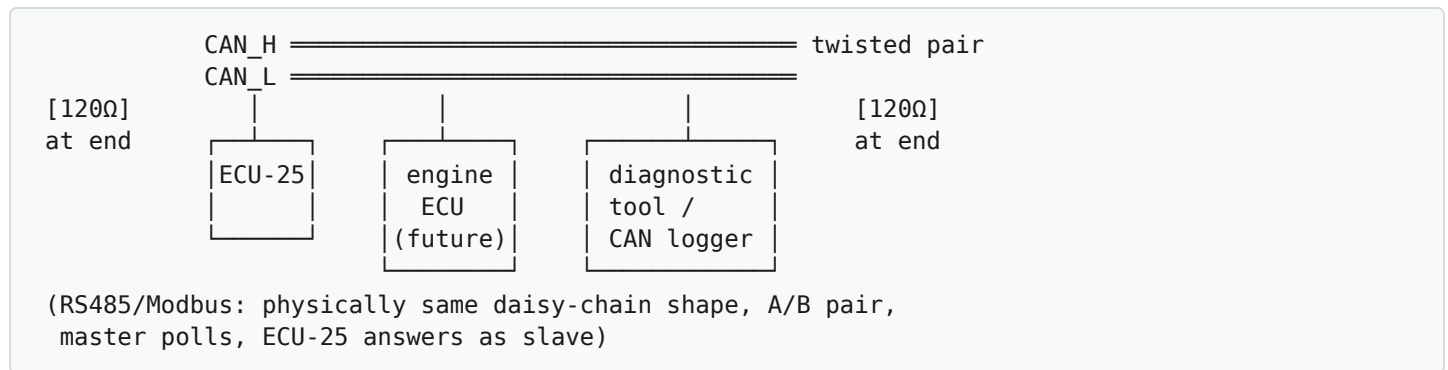
| Relay     | Function                        | Why it exists                                                                                                                                                                       |
|-----------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| K1 (16 A) | <b>Fuel solenoid</b>            | The energize-to-run valve on the injection pump. Powered = engine can run; released = engine stops. This is also the ultimate protection actuator: every shutdown ends with K1 off. |
| K2 (16 A) | <b>Starter motor</b>            | Drives the starter solenoid. Big inrush, hence 16 A rating. Firmware enforces crank time limits and crank disconnect.                                                               |
| K3        | <b>Generator contactor coil</b> | Energizes the big external contactor that connects the load to the generator. Contacts rated 250 VAC because contactor coils are usually 230 VAC.                                   |
| K4        | <b>Mains contactor coil</b>     | Same, for the mains side.                                                                                                                                                           |
| K5        | <b>Alarm / horn</b>             | Sounds on warning or shutdown.                                                                                                                                                      |
| K6        | <b>Preheat / glow</b>           | Drives glow-plug relay for cold starts (timed by firmware before cranking).                                                                                                         |

**The transfer interlock — belt and braces:** K3 and K4 must never be on together (that would parallel an unsynchronized generator with the mains — effectively a short between two out-of-phase sources; breakers trip, or worse). Protection is layered: (1) firmware interlocks the two outputs with a dead time (**break-before-make**); (2) the panel wiring must ALSO cross-interlock the two contactors electrically and, ideally, mechanically. Never trust software alone for this — a documented wiring requirement in the manual.

## 11. Communications

### 11.1 CAN bus + J1939

**CAN (Controller Area Network)** — the automotive field bus. Refresher: a two-wire differential bus (CAN\_H/CAN\_L), multi-master, up to 1 Mbit/s, with arbitration by message ID (lower ID wins, losers retry automatically — no collisions lost), hardware CRC and acknowledgement in every frame. Extremely robust in electrically noisy environments, which is why every vehicle uses it. The **TJA1051T/3** is the transceiver — the physical-layer chip converting the MCU's logic-level TX/RX to the differential bus levels (the /3 variant has 3.3 V-compatible I/O). The bus needs **120 Ω termination at each physical end** (we provide a jumpered split termination — two 60 Ω to a common-mode capacitor — enabled only when our node is a bus end).



**J1939** is the higher-layer protocol standardized on top of CAN for heavy vehicles and industrial engines (SAE J1939): 29-bit identifiers carrying a **PGN (Parameter Group Number)** — a message-type ID — and a source address; data fields are standardized **SPNs** (e.g. SPN 190 = engine speed) with defined scaling and offsets; nodes negotiate addresses by **address claiming**. Any J1939-literate device can decode standard PGNs from any manufacturer. ECU-25 claims an address and broadcasts genset/engine PGNs; on a future electronic engine it would also *read* the engine ECU's broadcasts instead of using analog senders.

## 11.2 RS485 + Modbus RTU

**RS-485** — the industrial two-wire differential serial standard: multi-drop (up to 32+ nodes on one twisted pair), robust over hundreds of metres. The **THVD1450** transceiver handles the physical layer; the board carries jumpered **termination** (120 Ω at cable ends) and **fail-safe bias resistors** (they pull the idle bus to a defined state so receivers don't chatter when nobody is transmitting).

**Modbus RTU** — the plain-vanilla industrial protocol running over that wire since 1979: a master (SCADA, PLC, laptop) polls slaves by address; slaves expose numbered 16-bit **registers** (read holding registers, write single register, etc.), CRC-16 per frame. ECU-25 is a Modbus **slave**: every measurement, state, and alarm is readable and every configuration parameter writeable through a documented **register map** ( docs/modbus-map.md , to be written). This is what makes the controller integrable into any building-management or SCADA system.

## 11.3 The spare PWM output

One MCU timer PWM pin, routed to the I/O connector unused. If a future engine has an **electronic governor** (an actuator moving the fuel rack, needing a PID speed loop instead of a mechanical flyweight governor), the control signal already exists at the connector — only an external actuator

driver stage is needed, no board respin. Reserving pins for known future features is standard practice; pins are free at design time and unobtainable afterwards.

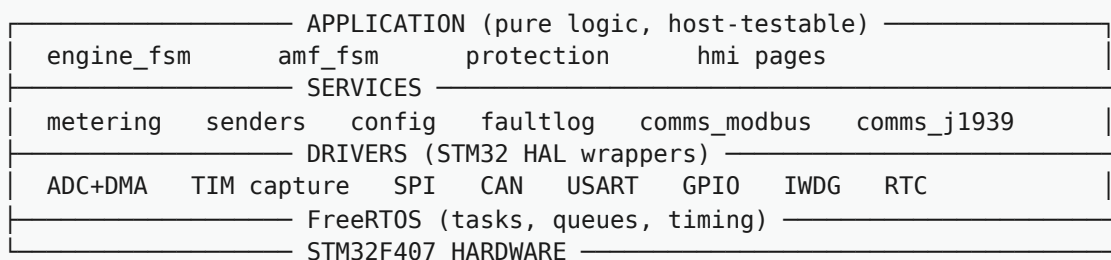
## 12. Firmware Architecture

Platform: **C**, STM32 **HAL** (ST's hardware abstraction library), **FreeRTOS** (a small real-time operating system providing preemptive tasks, queues, and timing), built with arm-none-eabi-gcc and CMake.

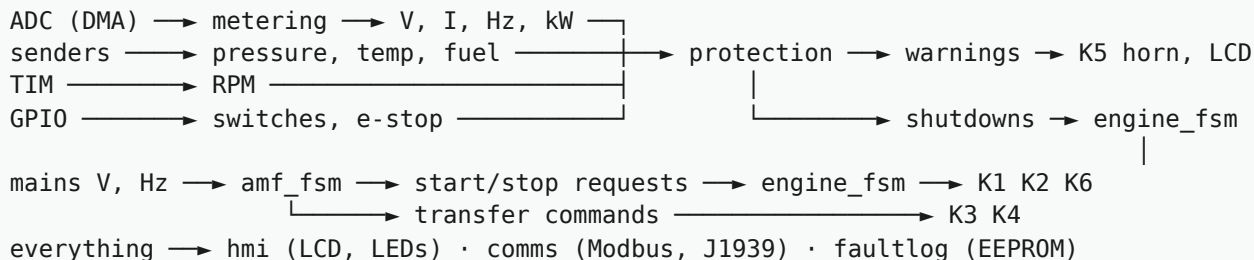
**Why an RTOS here?** The workload is a mix of hard-timed fast work (ADC streams, protection checks every 10 ms) and slow work (LCD redraws, Modbus polls). An RTOS lets each run as a prioritized task; a slow LCD redraw can never delay a protection check. It is also the architecture you'd meet in industry.

**Design rule: logic modules are pure.** The state machines and protection logic are written as plain C with no hardware calls — inputs in, decisions out. This lets us compile and unit-test them **on the host PC** (no board needed), which is how the tricky sequencing logic gets exhaustively tested.

Layered architecture — each layer only calls the one below it:



Data flow, once per decision cycle:



### 12.1 engine\_fsm — the start/stop state machine

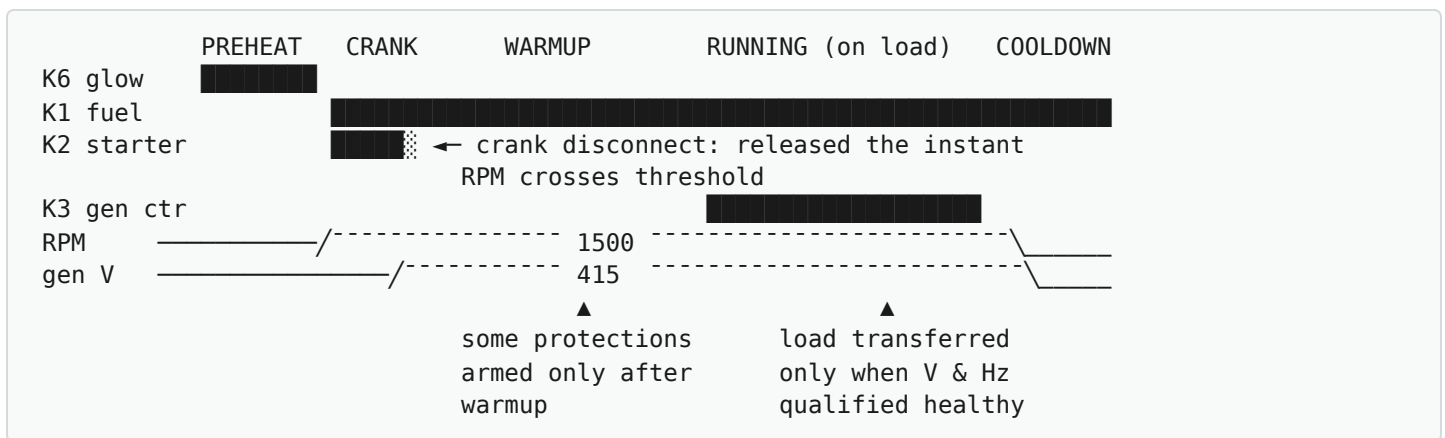
```
STOPPED → PREHEAT → CRANK → CRANK_REST → (retry ≤3) → SHUTDOWN(fail-to-start)
                                     ↓ (RPM ≥ threshold OR oil pressure rises)
RUNNING_WARMUP → RUNNING → COOLDOWN → STOPPING → STOPPED
```

- **PREHEAT:** K6 on for the configured glow time (cold diesels need it). K1 (run-enable) also comes on here — a J1939 engine ECU uses these seconds to boot, so live RPM is already on the bus when cranking starts; in legacy mode it just energizes the fuel solenoid early.



- **CRANK:** K2 (starter) + K1 (fuel) on, for max ~10 s. **Crank disconnect** — the moment the engine fires (RPM crosses ~500, or oil pressure switch opens), K2 releases instantly. This single feature is why RPM sensing must be reliable.
- **CRANK\_REST:** starter motors overheat; between attempts the FSM waits (~10 s). After 3 failed attempts: **fail-to-start shutdown** (a genset that cranks its battery flat is worse than one that alarms).
- **RUNNING\_WARMUP:** engine runs off-load briefly; some protections (like under-voltage) are held off until the set stabilizes.
- **COOLDOWN:** after load is removed, the engine idles a few minutes before stopping — turbochargers and injectors live longer when not stopped hot.
- **STOPPING:** K1 off; firmware verifies RPM actually falls to zero (**fail-to-stop** alarm if not — a runaway diesel is a real, dangerous failure).

A successful automatic start, as a timeline:



## 12.2 protection — the safety engine

Runs every 10 ms over every monitored quantity. Two classes:

- **Warning:** operator attention needed; alarm relay + display, engine keeps running (e.g. battery voltage low).
- **Shutdown:** immediate fuel cut + latched alarm (e.g. low oil pressure, high coolant temperature, overspeed, generator over-voltage). **Latched** = the alarm stays displayed after the engine stops, until a human presses STOP/reset — so the 3 a.m. failure is still on screen at 8 a.m., and the set cannot auto-restart into the same fault.

Each protection has: a threshold, a delay (to ride through transients — e.g. under-voltage must persist X seconds), an enable window (oil pressure is only meaningful once running), and a class. All configurable.

## 12.3 amf\_fsm — the mains-watching state machine

```
MAINS_HEALTHY → (mains fails, qualification delay) → STARTING_GEN
→ GEN_ON_LOAD → (mains returns, qualification delay) → RETRANSFER
→ COOLDOWN → STOPPED → MAINS_HEALTHY
```

Every transition is guarded by **qualification timers** — mains must be dead for N seconds before starting (ride through flickers), and healthy for M seconds (typically 30–60 s) before transferring back (utilities often return unstable). Transfer sequence: open one contactor, dead time, close the other — **break-before-make**, enforced in both firmware and panel wiring (§10).

## 12.4 metering — the numeric heart

- The triple ADC free-runs via **DMA** (direct memory access — the peripheral writes samples into RAM buffers with zero CPU involvement), sampling all AC channels **128 samples per cycle**, with the sample rate continuously adjusted to track the measured frequency (so one buffer = exactly one cycle even as engine speed varies).
- Per cycle: true RMS ( $\sqrt{\text{mean}(v^2)}$ ),  $P = \text{mean}(v \cdot i)$ , S, PF, frequency from interpolated zero-crossings. Averaged over several cycles for stable display, but protection comparisons use the fast values.
- Also maintained: kWh accumulator and the **engine hour-meter** (the number every maintenance schedule runs on), both persisted to EEPROM.

## 12.5 The rest

- **senders:** ADC counts → resistance → engineering units through config interpolation tables.
- **hmi:** LCD page rendering (status, metering, alarm list, config menus), key handling, LED map.
- **comms\_modbus / comms\_j1939:** as in §11, with documented external interfaces.
- **config:** versioned, CRC-guarded parameter block in EEPROM; sane defaults burned in for first boot.
- **faultlog:** circular event log in EEPROM — every alarm/shutdown with RTC timestamp, hour-meter reading, and a snapshot of key values (the black-box that answers "why did it trip last night?").

## 12.6 Timing and the honest watchdog

1 kHz base tick. Protection at 100 Hz. Metering continuous via DMA. HMI at 20 Hz. The IWDG (§4.1) is kicked **only when every critical task has checked in** during the window (each task sets its bit in an alive-bitmask; the lowest-priority task kicks the dog only on a full mask, then clears it). Kicking a watchdog from a timer interrupt regardless of system health — a common bad habit — makes the watchdog decorative.

---

## 13. Display Board

---

- **4.3" 480×272 color TFT with RA8875 graphics controller, SPI interface.** Phone-size, matching modern CPCB IV+ era controllers. The RA8875 is the trick that keeps this simple: it carries its own framebuffer RAM and a hardware drawing engine, so our MCU sends compact drawing commands over SPI instead of pushing every pixel of a raw RGB panel (which would demand a bigger MCU + external SDRAM). Backlight runs from the 5 V rail with PWM dimming.
  - **7 keys:** STOP, AUTO, MANUAL, START (the standard genset-controller four — STOP also resets latched alarms; AUTO arms the AMF logic; MANUAL/START give direct human control), plus ▲ ▼ ↵ for menus.
  - **8 LEDs:** Mains OK, Gen Running, Load-on-Mains, Load-on-Gen, Warning, Shutdown, Auto-mode, Charge.
  - **Shift registers** (74HC165 input for keys, 74HC595 output for LEDs) sit on the same SPI bus as the TFT controller with separate chip-selects — the entire panel needs a **20-way ribbon**: 3.3 V, 5 V (backlight), ground, SPI, RA8875 control lines, strobes. The UI itself is built with **LVGL**, whose PC simulator lets the whole interface be designed and approved on a desktop before any hardware exists.
- 

## 14. Connectors and Physical Design

---

- **Pluggable screw-terminal blocks, 5.08 mm pitch**, grouped by function: DC power | digital inputs | senders + RPM | relay contacts | AC voltage inputs | CTs | comms. Pluggable = the installer wires the plugs at the bench and the controller can be swapped in the field in minutes without rewiring. Grouping by function (and physically separating the AC group at one board edge) prevents wiring mistakes and keeps hazardous voltage away from signal wiring.
  - Display ribbon: 16-way boxed IDC (polarized shell — cannot be inserted reversed).
  - Display board sized toward a **96×96 mm DIN cutout** — the standard panel-meter hole, so the unit mounts in any standard genset panel door.
- 

## 15. Safety and Test Plan — Why Staged

---

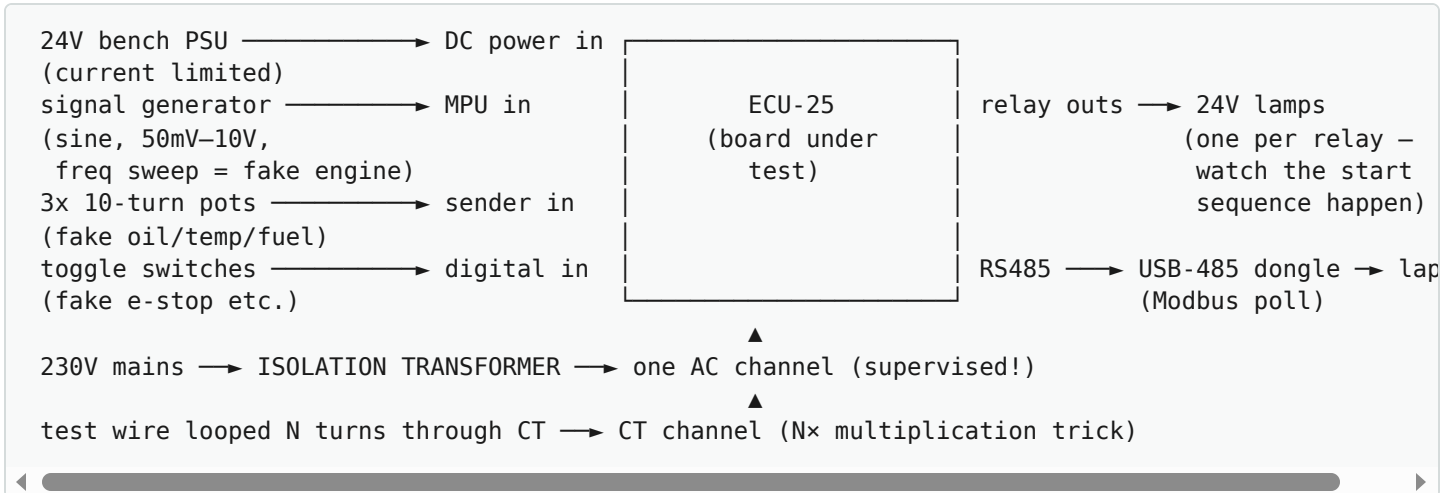
Never first-power a new board fully connected. The stages:

1. **Bring-up (no AC, no engine):** current-limited bench supply; verify rails, then MCU sign-of-life, then each peripheral in isolation. A solder bridge found here costs minutes; found on a genset it costs the board.
2. **Bench rig:** 24 V bench PSU; **signal generator pretending to be the MPU** (so the whole start sequence and overspeed logic run with no engine); potentiometers as senders; switches as digital inputs; lamps as relay loads; **one AC channel validated with 230 V through an isolation transformer** (an isolation transformer breaks the galvanic connection to the mains supply, so touching a single point of the secondary can't complete a circuit through you to earth

— the standard safe way to develop mains-connected circuits); CT math validated by looping a wire N turns through the CT so a small test current looks like  $N \times$  the current.

3. **Genset dry runs:** on the real engine with the **fuel solenoid deliberately held off** — cranking, crank-disconnect, e-stop verified with the engine never actually starting.
4. **Staged commissioning:** engine control alone → generator metering → live protections → AMF transfer last, each stage supervised.

The bench rig — the whole controller exercised with zero horsepower in the room:



**Non-negotiable rules:** all 415 V wiring done de-energized and verified dead; AC terminals shrouded; never handle the board while mains sensing is live (remember §8 — no galvanic isolation); the contactor hardware interlock is mandatory in the panel regardless of firmware.

## 16. Glossary (quick reference)

---

| Term                 | Meaning                                                                  |
|----------------------|--------------------------------------------------------------------------|
| ADC                  | Analog-to-digital converter — measures a voltage as a number             |
| AEC-Q100 / -Q1       | Automotive qualification standard for ICs / suffix marking it            |
| AMF                  | Auto Mains Failure — auto start & transfer on power cut                  |
| Burden               | The low-impedance load a CT secondary must always drive                  |
| bxCAN                | The STM32's built-in CAN controller peripheral                           |
| CAN                  | Controller Area Network — robust differential vehicle bus                |
| Clearance / Creepage | Safety distance through air / along surface between conductors           |
| CRC                  | Cyclic redundancy check — error-detecting checksum                       |
| CT                   | Current transformer — clamps a cable, scales its current down            |
| DMA                  | Direct memory access — peripherals move data without the CPU             |
| FSM                  | Finite state machine — logic organized as named states + transitions     |
| HAL                  | Hardware abstraction layer — ST's peripheral driver library              |
| IWDG                 | Independent watchdog — hardware reset if firmware hangs                  |
| J1939                | SAE standard protocol over CAN for engines/heavy vehicles                |
| LDO                  | Low-dropout linear regulator — clean rail, low headroom                  |
| Load dump            | Rail spike when battery disconnects under charge                         |
| LSE/HSE              | Low/high-speed external oscillator (32.768 kHz / 8 MHz crystals)         |
| Modbus RTU           | Register-based industrial serial protocol over RS-485                    |
| MPU                  | Magnetic pickup — passive coil RPM sensor at the flywheel                |
| NTC                  | Negative temperature coefficient thermistor                              |
| PGN/SPN              | J1939 message ID / standardized parameter within it                      |
| PSRR                 | Power-supply rejection ratio — how well a regulator rejects input ripple |
| PWM                  | Pulse-width modulation                                                   |
| RTC                  | Real-time clock (calendar time, battery/supercap backed)                 |
| Sender               | Resistive engine sensor (pressure/temperature/level)                     |
| SWD                  | Serial Wire Debug — 2-wire ARM flash/debug interface                     |
| True RMS             | $\sqrt{(\text{mean of squared samples})}$ — correct for any waveshape    |
| TVS                  | Transient voltage suppressor diode — clamps spikes                       |